Data Structures and Algorithms -3 (25CS2103E)

Project - Warehouse Inventory System

Team Members -

Santhosh Pendyala-2520030330
Haadi Hasil-2520030297

Sashank Chivukula-2520030405

Under the Esteemed Guidance of

DR. V Sireesha

Contents

- Abstract
- Introduction
- Literature survey
- Problem Statement and Objectives
- Hardware and Software requirements
- Results
- Conclusion
- Future Scope

Abstract

The Warehouse Inventory System is a data-structure-driven application built to manage product inventory across large-scale warehouses efficiently. It tackles three core challenges — fast product lookup, low-stock detection, and supplier information retrieval — using hash tables for constant-time SKU/name lookups, balanced search trees (BST/AVL) for sorted, range-based queries, and a min-heap for instantly identifying items needing reorder without scanning the full inventory.

Supplier relationships are modeled as a graph, enabling quick traversal to find alternate suppliers or shortest procurement paths during critical stock shortages. By combining hashing, trees, heaps, and graphs, the system replaces linear search and manual tracking with logarithmic and constant-time operations, significantly improving scalability and responsiveness for real-world inventory management.

Introduction

Managing inventory in a large warehouse gets messy fast. With thousands of products, manually tracking stock levels, finding items, and reaching out to suppliers becomes slow and error-prone — and it only gets worse as inventory grows. This project sets out to fix that.

The issue: Manual search and tracking don't scale with large inventories.

The impact: Leads to delays, missed low-stock alerts, and slower supplier response during shortages.

The goal: Enable instant product lookup, automatic low-stock detection, and quick supplier access.

The approach: Use hash tables for lookup, trees for sorted search, a heap for stock alerts, and a graph for supplier connections.

The outcome: A faster, more reliable inventory system that scales smoothly.

Literature Survey

Study/System	Approach Used	Limitation
Traditional WMS (SQL-based)	Relational DB queries	Slower for real-time large-scale lookups without heavy indexing
Basic inventory scripts	Linear Search/Arrays	O(n) lookup, doesn't scale
RFID-based tracking systems	Hardware tracking + DB	Good for physical tracking, no algorithmic stock-priority logic

Problem Statement and Objectives

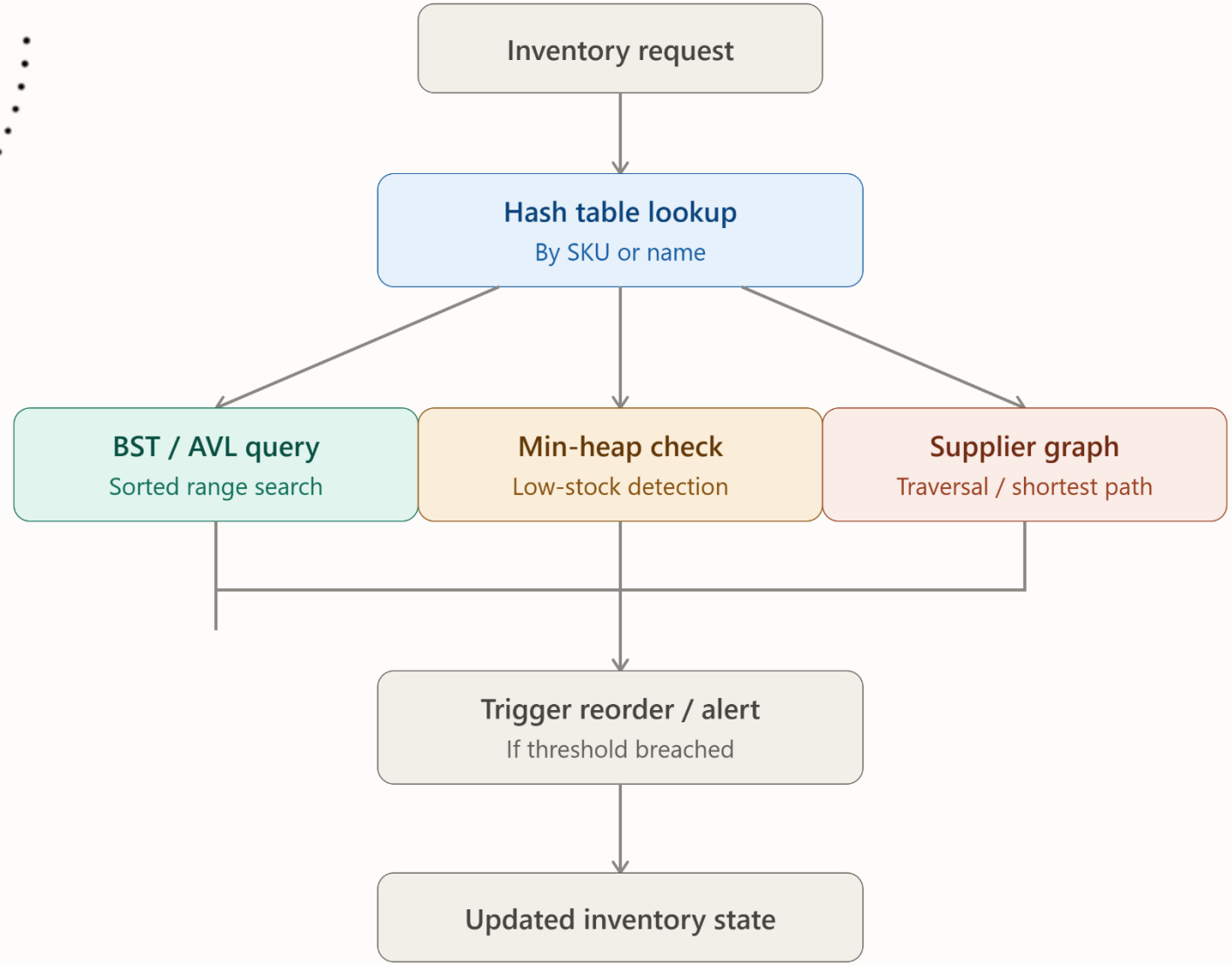
Current warehouse inventory systems often face:

- Slow product searching in large inventories
- Difficulty identifying low-stock items quickly
- Inefficient supplier management
- Time-consuming manual inventory tracking

The proposed Functionalities are :

- Product search using SKU or Product Name
- Inventory management (Add, Update, Delete Products)
- Automatic low-stock detection
-
- Supplier information management

Flow Chart


Inventory request
Hash table lookup
By SKU or name
BST / AVL query
Sorted range search
Min-heap check
Low-stock detection
Supplier graph
Traversal / shortest path
Trigger reorder / alert
If threshold breached
Updated inventory state

Results

Operation	Traditional (linear / manual)	Implemented (this project)	Improvement
Product lookup by SKU/name	O(n)	O(1) — hash table	Constant time
Range-based stock query	O(n)	O(log n) — BST/AVL	Logarithmic
Low-stock identification	O(n) full scan	O(1) — min-heap top	Instant
Finding alternate supplier	Manual lookup	O(V+E) — graph traversal	Automated, fast
Shortest procurement path	Not available	O((V+E) log V) — Dijkstra	New Capability

Hardware and Software requirements

Hardware requirements :

- Processor: Intel i3 / AMD Ryzen 3 or higher (dual-core minimum)
- RAM: 4 GB minimum (8 GB recommended for large dataset testing)
- Storage: 500 MB free disk space
- Standard I/O devices: keyboard, mouse, monitor

Software requirements :

- Operating System : Windows 10/11
- Programming Language : Java
- Git
- Visual Studio Code

Conclusion

- The Warehouse Inventory System successfully demonstrates how core data structures — hash tables, BST/AVL trees, min-heaps, and graphs — can solve real-world inventory management problems efficiently.
- Replacing linear search and manual tracking with these structures significantly improves lookup speed, low-stock detection, and supplier coordination.
- The system scales well as inventory size grows, making it suitable for large-scale warehouse operations.
- This project reinforces practical understanding of classical DSA concepts by applying them to a cohesive, real-world use case rather than isolated exercises.

Future Scope

- Database integration — persistent storage instead of in-memory data.
- GUI/dashboard — visual interface for staff to view stock and alerts.
- Real-time supplier APIs — live pricing and availability instead of static data.
- Multi-warehouse support — extend the graph to handle inter-warehouse transfers.
- Predictive reordering — forecast demand using historical trends.
- Concurrency & access control — safe multi-user access with role-based permissions.

(DEEMED TO BE UNIVERSITY)

Thank you